



Informe TP-2

Grupo 15

Teoría de Algoritmos I
Cátedra Echeverría

Integrantes:

Apellido y Nombre	Padrón	Mail
Zysman Bayetto, Nicolás	108932	nzysman@fi.uba.ar
Gentilini, Franco	108733	fgentilini@fi.uba.ar
Retamozo, Melina	110065	mrretamozo@fi.uba.ar
Areco Gonzalez, Lucas	111203	lareco@fi.uba.ar

Fecha de entrega: 21/06/2026

Índice

1. Problema 1 – Programación Lineal	2
1.1. Problema a Resolver	2
1.2. Supuestos	2
1.3. Variables de Decisión	2
1.4. Modelo de Programación Lineal Entera Binaria	2
1.5. Solución Computacional	3
2. Problema 2 – Redes de Flujo	3
2.1. Problema a resolver	3
2.2. Supuestos	3
2.3. Diseño	4
2.4. Pseudocódigo	5
2.5. Seguimiento	6
2.6. Análisis de complejidad	7
2.7. Sets de datos y validación	8
2.8. Tiempos de ejecución	9
3. Problema 3 – Algoritmos de Aproximación	9
3.1. Problema a resolver	9
3.1.1. Algoritmo inicial	9
3.2. Contraejemplo del algoritmo inicial	10
3.3. Algoritmo propuesto	10
3.4. Garantía de calidad	10
3.5. Pseudocódigo	11
3.6. Estructuras de datos utilizadas	11
3.7. Seguimiento	11
3.8. Complejidad	12
3.9. Set de datos y tiempos de ejecución	12
3.10. Resultados	14
4. Problema 4 – Algoritmos Aleatorios	14
4.1. Problema a Resolver	14
4.2. Supuestos	15
4.3. Algoritmo y Estructura de Datos	15
4.4. Pseudocódigo	15
4.5. Seguimiento de Ejemplo	15
4.6. Demostración	17
4.7. Análisis de Complejidad	17
4.8. Informe de Resultados	17

1. Problema 1 – Programación Lineal

1.1. Problema a Resolver

Concesiones Argentina 2000 SRL dispone de 200 paradas de colectivo para vender espacios publicitarios durante un mes. Cada cliente ofrece un monto fijo a cambio de ocupar cierta cantidad de paradas. Se debe decidir qué ofertas aceptar para maximizar el beneficio total, respetando:

- la capacidad total de 200 paradas,
- que para el cliente B solo puede elegirse una de sus dos opciones,
- y que los clientes A y D no pueden pautar simultáneamente por competencia directa.

1.2. Supuestos

- Cada oferta se acepta o se rechaza completamente (no se puede fraccionar una oferta).
- Si se acepta una oferta, se cobran exactamente los USD indicados en el enunciado.
- Todas las paradas son homogéneas para el modelo (solo importa cuántas se ocupan, no cuáles).
- La demanda de cada cliente está garantizada: si se selecciona una oferta, puede implementarse sin restricciones adicionales.

1.3. Variables de Decisión

Se definen variables binarias (indicativas):

- x_A : vale 1 si se acepta la oferta del cliente A (30 paradas por USD 50000), y 0 en caso contrario.
- x_{B80} : vale 1 si se acepta la opción de B de 80 paradas por USD 100000, y 0 en caso contrario.
- x_{B120} : vale 1 si se acepta la opción de B de 120 paradas por USD 120000, y 0 en caso contrario.
- x_C : vale 1 si se acepta la oferta de C (75 paradas por USD 100000), y 0 en caso contrario.
- x_D : vale 1 si se acepta la oferta de D (50 paradas por USD 80000), y 0 en caso contrario.
- x_E : vale 1 si se acepta la oferta de E (2 paradas por USD 5000), y 0 en caso contrario.
- x_F : vale 1 si se acepta la oferta de F (20 paradas por USD 40000), y 0 en caso contrario.
- x_G : vale 1 si se acepta la oferta de G (100 paradas por USD 90000), y 0 en caso contrario.

Todas las variables son binarias: $x_i \in \{0, 1\}$.

1.4. Modelo de Programación Lineal Entera Binaria

Función objetivo (maximizar beneficio):

$$\text{máx } Z = 50000x_A + 100000x_{B80} + 120000x_{B120} + 100000x_C + 80000x_D + 5000x_E + 40000x_F + 90000x_G$$

Restricciones:

- Capacidad total de paradas:

$$30x_A + 80x_{B80} + 120x_{B120} + 75x_C + 50x_D + 2x_E + 20x_F + 100x_G \leq 200$$

- Exclusión de alternativas del cliente B:

$$x_{B80} + x_{B120} \leq 1$$

- Incompatibilidad entre A y D:

$$x_A + x_D \leq 1$$

- Dominio de las variables:

$$x_A, x_{B80}, x_{B120}, x_C, x_D, x_E, x_F, x_G \in \{0, 1\}$$

1.5. Solución Computacional

El modelo se resolvió en Python con la librería PuLP y el solver CBC.

Selección óptima obtenida:

- A: seleccionado ($x_A = 1$)
- B (opción 80 paradas): seleccionado ($x_{B80} = 1$)
- C: seleccionado ($x_C = 1$)
- E: seleccionado ($x_E = 1$)
- Resto de ofertas: no seleccionadas

Valor óptimo:

- Beneficio total: USD 255000
- Paradas utilizadas: 187 de 200

Por lo tanto, la mejor decisión para el mes es concesionar publicidad a los clientes A, B (80), C y E.

2. Problema 2 – Redes de Flujo

2.1. Problema a resolver

Estamos construyendo una red WAN con n antenas y queremos que tenga un buen nivel de tolerancia a fallas. Dada una antena, su conjunto de backup de tamaño k es el conjunto de k antenas que se encuentran a una distancia menor a D . Queremos evitar que una antena pertenezca al conjunto de backup de más de b antenas, precisamente para evitar que un fallo pueda afectar a una porción importante de la red. Suponer que conocemos los valores D , b y k , y que tenemos una matriz $d[1..n, 1..n]$ con las distancias entre antenas, de forma tal que $d[i, j]$ es la distancia entre la antena i y la j . Plantear un algoritmo de complejidad polinomial que encuentre el conjunto de backup de tamaño k de cada una de las n antenas, de forma tal que ninguna aparezca en más de b conjuntos de backup, o bien, que indique que no existe una solución posible.

2.2. Supuestos

- La matriz de distancias es cuadrada de tamaño $n \times n$.
- Las distancias y los valores de entrada son todas positivas.
- Una antena no puede ser backup de sí misma.
- Una antena j solo puede actuar como backup de una antena i si $distancias[i][j] < D$.
- Cada antena debe recibir exactamente k antenas de backup.
- Cada antena candidata puede respaldar como máximo a b antenas.
- Se asume que $k \leq b$. En caso contrario, el algoritmo finaliza inmediatamente devolviendo NULO.
- Una solución es considerada válida únicamente cuando el flujo máximo obtenido es igual a $n \cdot k$.

2.3. Diseño

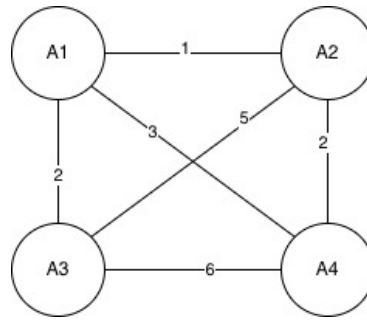


Figura 1: estado inicial de la red de antenas

Conceptualmente, cualquier antena podría ser backup de cualquier otra, pero la condición de distancia D restringe ese conjunto de candidatos (Figura 1). Por lo tanto, se modela este grafo como una red de flujo. Se construye un grafo dirigido con capacidades a partir de los datos de entrada:

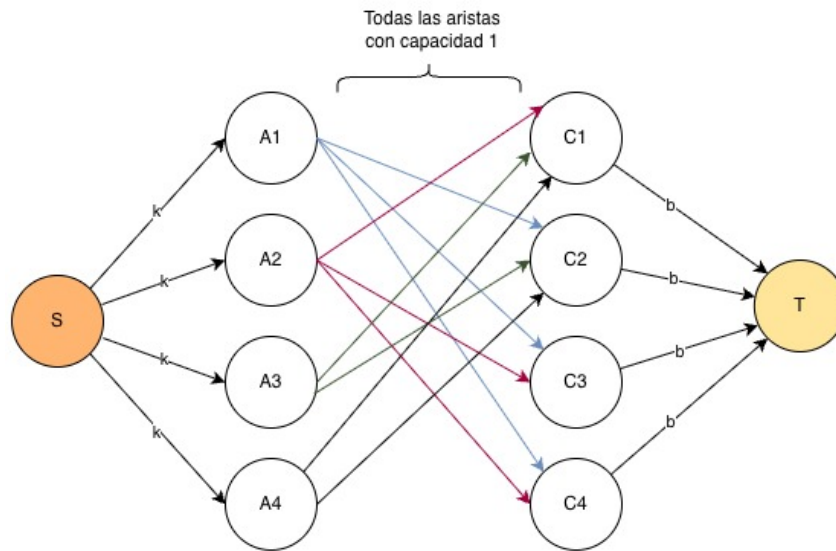


Figura 2: red de flujo construida a partir de los datos de entrada

En primer lugar, la fuente S se conecta a cada vértice que representa una antena con una arista de capacidad k , debido a que cada antena necesita exactamente k unidades de flujo para completar su conjunto de backup. Para cada uno de estos vértices antena i se conecta a un vértice antena candidata j con una arista de capacidad 1, únicamente si la distancia entre i y j es menor a D e i es distinto de j . Esta arista representa la posibilidad de que la antena j forme parte del backup de la antena i . La capacidad 1 garantiza no repetir una antena candidata j en el mismo backup de la antena i . Luego, cada vértice de antena candidata j se conecta directamente al sumidero T mediante una arista de capacidad b . Esta capacidad representa la restricción de que una antena puede pertenecer como máximo a b conjuntos de backup distintos. Como toda unidad de flujo que atraviesa el vértice candidata j debe necesariamente pasar por esta arista para llegar al sumidero, nunca podrán circular más de b unidades de flujo asociadas a dicha antena.

Sobre esta red de flujo se aplica el algoritmo de Ford-Fulkerson utilizando la estrategia de búsqueda de caminos aumentantes de Edmonds-Karp, que busca los caminos de la fuente al sumidero utilizando, según Cornell University (2010), BFS y garantiza la polinomialidad. Para afirmar que existe solución, el valor de flujo obtenido debe ser igual a $n * k$, ya que esa es la única forma en que todas las antenas puedan haber completado efectivamente sus k backups (si alguna antena no llegara a completar sus k , la suma total sería menor a $n * k$). Además, si $n * k$ es igual al flujo máximo obtenido, se desea conocer cómo quedó conformado el conjunto de backup de cada antena. Para ello, se realiza una reconstrucción a partir del grafo residual final. En particular, para cada vértice antena i , se identifican aquellos vértices

candidata j tales que la arista inversa (j, i) posee capacidad residual 1. Esto indica que una unidad de flujo fue enviada desde i hacia j durante la ejecución del algoritmo y, por lo tanto, que la antena j fue seleccionada para formar parte del conjunto de backup de la antena i . Repitiendo este procedimiento para las n antenas, se obtiene el conjunto de backup completo de tamaño k para cada una de ellas.

2.4. Pseudocódigo

```
FUNCION ARMAR_RED_FLUJO( $n, D, distancias, k, b$ )
  red = inicializar grafo con vértice fuente
  red[fuente][ant_i] =  $k$  para cada  $i$  en 1 a  $n$ 
  red[ant_i][cand_j] = 1 para cada  $i, j$  en 1 a  $n$  con  $i \neq j$  y  $distancias[i][j] < D$ 
  red[cand_j][sumidero] =  $b$  para cada  $j$  en 1 a  $n$ 

  DEVOLVER red

FUNCION CONVERTIR_A_GRAFO_RESIDUAL( $n, D, distancias, red$ )
  red[ant_i][fuente] = 0 para cada  $i$  en 1 a  $n$ 
  red[cand_j][ant_i] = 0 para cada  $i, j$  en 1 a  $n$  con  $i \neq j$  y  $distancias[i][j] < D$ 
  red[sumidero][cand_i] = 0 para cada  $i$  en 1 a  $n$ 

  DEVOLVER red

FUNCION OBTENER_FLUJO_MAX(residual,  $n, fuente, sumidero$ )
  camino = OBTENER_CAMINO_MINIMO(residual, fuente, sumidero)
  MIENTRAS camino no sea nulo:
    capacidad_min = INFINITO
    PARA CADA par de nodos consecutivos  $(u, v)$  en camino:
      SI residual[ $u$ ][ $v$ ] < capacidad_min:
        capacidad_min = residual[ $u$ ][ $v$ ]

    PARA CADA par de nodos consecutivos  $(u, v)$  en camino:
      residual[ $u$ ][ $v$ ] = residual[ $u$ ][ $v$ ] - capacidad_min
      residual[ $v$ ][ $u$ ] = residual[ $v$ ][ $u$ ] + capacidad_min

  camino = OBTENER_CAMINO_MINIMO(residual, fuente, sumidero)

  flujo_max = suma de residual[ant_i][fuente] para  $i = 1$  hasta  $n$ 
  RETORNAR flujo_max, residual

FUNCION OBTENER_BACKUPS(red,  $n$ ):
  backups = diccionario vacío
  PARA CADA antena  $i$  HASTA  $n$ 
    backup_i = list vacía
    PARA CADA candidato  $j$  desde 1 HASTA  $n$ :
      SI  $i \neq j$  Y el candidato  $j$  esta en candidatos de antena  $i$ :
        SI la capacidad entre candidato  $j$  y antena  $i$  es 1:
          AGREGAR candidato  $j$  a backup_i
  DEVOLVER backups

FUNCION RESOLVER_BACKUPS( $D, distancias, k, b$ ):
  SI  $k > b$ :
    DEVOLVER NULO
   $n = \text{LONGITUD DISTANCIAS}$ 
  red = ARMAR_RED_FLUJO( $n, D, distancias, k, b$ )
  residual = CONVERTIR_A_GRAFO_RESIDUAL( $n, D, distancias, red$ )
  flujo_max, residual = OBTENER_FLUJO_MAX(residual,  $n, "s", "t")$ 
  SI  $n * k$  es igual a flujo_max:
    DEVOLVER OBTENER_BACKUPS(residual,  $n$ )
  DEVOLVER NULO
```

En la implementación, la red de flujo y el grafo residual se representan mediante diccionarios de diccionarios, donde cada clave corresponde a un vértice y cada valor es otro diccionario que almacena sus vecinos junto con la capacidad de la arista correspondiente. Esto permite acceder y modificar capacidades en tiempo constante.

La red de flujo y el grafo residual se representan mediante diccionarios de diccionarios, equivalentes a listas de adyacencia con capacidades asociadas a cada arista.

Durante la ejecución de Edmonds-Karp se utilizan las siguientes estructuras auxiliares:

- Un diccionario **padres**, que almacena para cada vértice el vértice desde el cual fue alcanzado durante el BFS. Se utiliza para reconstruir caminos aumentantes.
- Una lista **visitados**, utilizada para evitar visitar un mismo vértice más de una vez durante cada BFS.
- Una cola, utilizada por el BFS para recorrer la red por niveles y garantizar la búsqueda de caminos aumentantes con la menor cantidad de aristas.
- Una lista para representar cada camino aumentante reconstruido.

Una vez obtenido el flujo máximo, se utiliza un diccionario **backups** para almacenar la solución. Para cada antena se guarda la lista de antenas que conforman su conjunto de backup reconstruido a partir del grafo residual final.

2.5. Seguimiento

Se utiliza el siguiente conjunto de datos:

```
distancias = [  
    [0, 1, 1, 10],  
    [1, 0, 10, 1],  
    [1, 10, 0, 10],  
    [10, 1, 10, 0]]  
k = 1  
b = 1  
D = 2
```

Red de flujo inicial

```
s: {'a1': 1, 'a2': 1, 'a3': 1, 'a4': 1}  
a1: {'c2': 1, 'c3': 1}  
a2: {'c1': 1, 'c4': 1}  
a3: {'c1': 1}  
a4: {'c2': 1}  
c1: {'t': 1}  
c2: {'t': 1}  
c3: {'t': 1}  
c4: {'t': 1}  
t: {}
```

Se transforma la red de flujo en un grafo residual

```
s: {'a1': 1, 'a2': 1, 'a3': 1, 'a4': 1}  
a1: {'c2': 1, 'c3': 1, 's': 0}  
a2: {'c1': 1, 'c4': 1, 's': 0}  
a3: {'c1': 1, 's': 0}  
a4: {'c2': 1, 's': 0}  
c1: {'t': 1, 'a2': 0, 'a3': 0}  
c2: {'t': 1, 'a1': 0, 'a4': 0}  
c3: {'t': 1, 'a1': 0}  
c4: {'t': 1, 'a2': 0}  
t: {'c1': 0, 'c2': 0, 'c3': 0, 'c4': 0}
```

Se busca el flujo máximo aplicando el algoritmo de Edmonds-Karp

Iteracion 1

Camino: $s \rightarrow a1 \rightarrow c2 \rightarrow t$

Flujo enviado: 1

Aristas directas:

$(s, a1): 1 \rightarrow 0, (a1, c2): 1 \rightarrow 0, (c2, t): 1 \rightarrow 0$

Aristas inversas:

$(a1, s): 0 \rightarrow 1, (c2, a1): 0 \rightarrow 1, (t, c2): 0 \rightarrow 1$

Iteracion 2

Camino: $s \rightarrow a2 \rightarrow c1 \rightarrow t$

Flujo enviado: 1

Aristas directas:

$(s, a2): 1 \rightarrow 0, (a2, c1): 1 \rightarrow 0, (c1, t): 1 \rightarrow 0$

Aristas inversas:

$(a2, s): 0 \rightarrow 1, (c1, a2): 0 \rightarrow 1, (t, c1): 0 \rightarrow 1$

Iteracion 3

Camino: $s \rightarrow a3 \rightarrow c1 \rightarrow a2 \rightarrow c4 \rightarrow t$

Flujo enviado: 1

Aristas directas:

$(s, a3): 1 \rightarrow 0, (a3, c1): 1 \rightarrow 0, (c1, a2): 1 \rightarrow 0, (a2, c4): 1 \rightarrow 0, (c4, t): 1 \rightarrow 0$

Aristas inversas:

$(a3, s): 0 \rightarrow 1, (c1, a3): 0 \rightarrow 1, (a2, c1): 0 \rightarrow 1, (c4, a2): 0 \rightarrow 1, (t, c4): 0 \rightarrow 1$

Iteracion 4

Camino: $s \rightarrow a4 \rightarrow c2 \rightarrow a1 \rightarrow c3 \rightarrow t$

Flujo enviado: 1

Aristas directas:

$(s, a4): 1 \rightarrow 0, (a4, c2): 1 \rightarrow 0, (c2, a1): 1 \rightarrow 0, (a1, c3): 1 \rightarrow 0, (c3, t): 1 \rightarrow 0$

Aristas inversas:

$(a4, s): 0 \rightarrow 1, (c2, a4): 0 \rightarrow 1, (a1, c2): 0 \rightarrow 1, (c3, a1): 0 \rightarrow 1, (t, c3): 0 \rightarrow 1$

Estas capacidades indican la cantidad de flujo que fue enviado desde la fuente hacia cada nodo a_i . Por lo tanto, el flujo máximo obtenido es:

Capacidad de $(a1, s) = 1$

Capacidad de $(a2, s) = 1$

Capacidad de $(a3, s) = 1$

Capacidad de $(a4, s) = 1$

Obtención de la solución Por lo tanto, el flujo total obtenido con el algoritmo de Edmonds-Karp es:

$$\text{flujo máximo} = 1 + 1 + 1 + 1 = 4.$$

Como el flujo máximo coincide con la cantidad total de copias de backup requeridas,

$$n \cdot k = 4 \cdot 1 = 4,$$

se concluye que existe una solución factible para el problema. Para obtener los conjuntos de backup, se observan en el grafo residual final las aristas inversas $c_j \rightarrow a_i$ con capacidad positiva. Cada una de estas aristas indica que la asignación correspondiente $a_i \rightarrow c_j$ fue utilizada por el flujo máximo. De esta forma se obtienen los siguientes conjuntos de backup:

$$\{a_1 : [a_3], a_2 : [a_4], a_3 : [a_1], a_4 : [a_2]\}$$

Puede verificarse que cada antenna posee exactamente $k = 1$ respaldos y que ninguna antenna pertenece a más de $b = 1$ backups, por lo que todas las restricciones del problema quedan satisfechas.

2.6. Análisis de complejidad

La red de flujo construida posee, en el peor de los casos (cuando todas las antenas se encuentran a una distancia menor a D de todas las demás), n aristas entre la fuente y las antenas, $n(n-1)$ aristas

entre las antenas y sus posibles candidatas, y n aristas entre las candidatas y el sumidero, dando un total de $n^2 + n$ aristas. Considerando además las aristas reversas incorporadas al construir el grafo residual, la cantidad total de aristas asciende a $2n^2 + 2n$. Asimismo, la red posee $2n + 2$ vértices: la fuente, un nodo para cada antena, un nodo por cada candidata y el sumidero.

La construcción de la red de flujo tiene complejidad temporal $O(n^2)$, dado que recorre todas las parejas posibles (i, j) para determinar si existe una arista entre la antena a_i y el candidato c_j . Los restantes recorridos involucrados en esta construcción son lineales y no modifican el orden de complejidad. De la misma forma, convertir la red de flujo a un grafo residual también recorre todas las parejas posibles (i, j) que puedan generar una arista entre un candidato y una antena, resultado en complejidad temporal $O(n^2)$.

En el peor de los casos el camino encontrado por BFS puede recorrer todos los vértices del grafo, dado que algunas iteraciones del algoritmo requieren transitar aristas reversas para reasignar antenas que quedaron sin candidatos disponibles. Por lo tanto, la reconstrucción de un camino tiene una complejidad temporal de $O(V) = O(2n + 2) = O(n)$. La búsqueda del camino más corto mediante BFS sobre el grafo residual tiene una complejidad de $O(V + E) = O(2n + 2 + 2n^2 + 2n) = O(2n^2 + 4n + 2) = O(n^2)$.

Sumando la complejidad de la reconstrucción del camino $O(n)$, que queda dominada por la búsqueda en sí, la complejidad total de la función `OBTENER_CAMINO_MINIMO` resulta $O(n^2)$.

La función `OBTENER_FLUJO_MAX` ejecuta un ciclo que se repite mientras exista un camino de fuente a sumidero en el grafo residual. Cada iteración en este ciclo realiza tres operaciones: la búsqueda del camino mediante `OBTENER_CAMINO_MINIMO` (complejidad $O(n^2)$, según se demostró previamente), y dos recorridos sobre el camino encontrado, uno para calcular la capacidad mínima del camino y otro para actualizar las capacidades residuales de sus aristas, cada uno con complejidad $O(n)$, dado que el camino puede involucrar hasta $2n + 2$ vértices. Como $O(n^2)$ domina sobre $O(n)$, el costo de una sola iteración del ciclo es $O(n^2)$.

Para acotar la cantidad total de iteraciones del ciclo, se analiza cuántas veces puede repetirse el siguiente proceso para una arista individual del grafo: la arista queda sin capacidad disponible luego de ser utilizada en una iteración, y posteriormente recupera capacidad disponible cuando una iteración distinta transita su arista reversa correspondiente. Este proceso no puede repetirse para siempre ya que cada arista solo puede pasar por este ciclo de usarse y recuperarse una cantidad limitada de veces, como máximo $2n + 2$, debido a que ese es el total de vértices que tiene la red.

Dado que la red posee $2n^2 + 2n$ aristas en total, y cada una puede repetir dicho proceso como máximo $2n + 2$ veces, la cantidad total de iteraciones del ciclo resulta: $O(n^2) * O(n) = O(n^3)$. Por lo tanto, la complejidad total de `OBTENER_FLUJO_MAX` se obtiene multiplicando el costo de cada iteración por la cantidad total de iteraciones del ciclo: $O(n^3) * O(n^2) = O(n^5)$.

Finalmente, la función `RESOLVER_BACKUPS` combina las funciones anteriores junto con `OBTENER_BACKUPS`, que determina la composición del backup de cada antena en tiempo $O(n^2)$. Dado que $O(n^5)$ domina a todas las demás complejidades involucradas, la complejidad temporal total del algoritmo resulta: $O(n^5)$. Al ser esta complejidad polinomial respecto a la cantidad de antenas n , se cumple el requisito de complejidad polinomial solicitado.

2.7. Sets de datos y validación

Caso	D	k	b	Resultado
caso_n4_con_solucion_k1	2	1	3	{a1: [2], a2: [1], a3: [1], a4: [2]}
caso_sin_solucion_k_mayor_b	5	3	2	None
caso_sin_solucion_por_D	2	2	3	None
caso_n4_con_solucion_k2	11	2	3	{a1: [2,3], a2: [1,3], a3: [1,2], a4: [1,2]}
caso_n6_con_solucion_k2	4	2	3	{a1: [2,3], a2: [1,3], a3: [1,2], a4: [1,2], a5: [3,4], a6: [4,5]}

Cuadro 1: Resultados obtenidos para cada set de datos manual.

2.8. Tiempos de ejecución

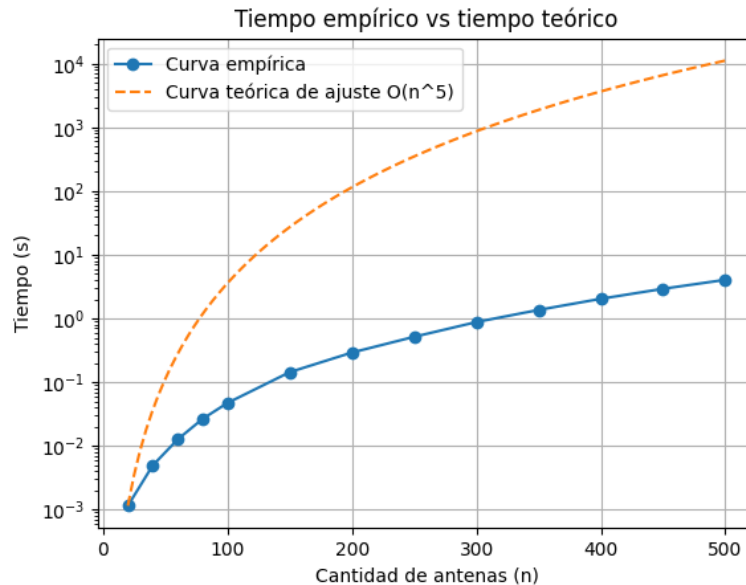


Figura 3: tiempo empírico contra tiempo teórico

Para comparar la curva empírica con la curva teórica de ajuste, se utilizó una escala logarítmica únicamente en el eje de tiempos, manteniendo el eje de cantidad de antenas en escala lineal. Esta decisión responde a la gran diferencia de magnitud entre ambas curvas: mientras que los tiempos medidos empíricamente son de pocos segundos, la curva teórica de ajuste alcanza valores de magnitudes mayores. Si ambos ejes se mantuvieran en escala lineal, esta diferencia haría que la curva empírica se viera aplastada contra el eje x, perdiendo toda visibilidad del crecimiento.

En cuanto a la interpretación de las curvas, en esta escala la pendiente de cada una refleja qué tan rápido crece su exponente: cuanto más pronunciada es la curvatura hacia arriba, mayor es el exponente de crecimiento. Se observa que la curva teórica, correspondiente a $O(n^5)$, presenta un ascenso considerablemente más marcado que la curva empírica, lo cual es consistente con que el algoritmo implementado crece, en la práctica, con un exponente menor al de la cota de peor caso. Como resultado, la curva empírica se mantiene siempre por debajo de la curva teórica en todo el rango de valores de n evaluados, evidenciando que el comportamiento real del algoritmo es considerablemente más eficiente que el escenario más desfavorable contemplado por el análisis de complejidad.

3. Problema 3 – Algoritmos de Aproximación

3.1. Problema a resolver

Tenemos un conjunto de enteros positivos $A = \{a_1, a_2, \dots, a_n\}$ y otro entero positivo B . Llamaremos subconjunto factible a un subconjunto S de A tal que la suma de sus elementos sea a lo sumo el valor B .

Queremos encontrar un subconjunto factible S de A tal que la suma de sus elementos sea la mayor posible. Por ejemplo, si $A = \{9, 3, 5\}$ y $B = 13$, entonces la solución óptima es $S = \{9, 3\}$

3.1.1. Algoritmo inicial

```

subconjunto_factible(A, B)
  S={ }
  T=0
  Para i=1,2,...,n
    Si T + A[i] <= B, entonces:
      S = S union A[i]
      T = T + A[i]

```

```

    fin si
  fin para
devolver S

```

3.2. Contraejemplo del algoritmo inicial

El algoritmo inicial presentado por el enunciado no asegura optimalidad pero se pueden encontrar múltiples contraejemplos que demuestran que las soluciones encontradas son peores que la mitad de una solución factible.

Para la siguiente instancia $A = \{10, 1, 40, 8\}$ y $B = 40$ la solución óptima sería $S = \{40\}$. Sin embargo el algoritmo inicial recorre ordenadamente el arreglo y agrega todos los elementos que no superen el valor B , por lo tanto el seguimiento sería:

```

S = {} T = 0 A={10, 1, 40, 8} B=40
(i=1)
T + A[i] = 10 < 40 -> S = {10} T = 10
(i=2)
T + A[i] = 11 < 40 -> S = {10, 1} T = 11
(i=3)
T + A[i] = 51 > 40 -> S = {10, 1} T = 11
(i=4)
T + A[i] = 19 < 40 -> S = {10, 1, 8} T = 19

```

Por incluir valores más pequeños primero, este algoritmo descarta el valor exacto B ya que incluirlo implica llegar a una solución incompatible.

3.3. Algoritmo propuesto

Este problema presentado es la versión de optimización del Subset Sum Problem que consiste en seleccionar un subconjunto de elementos cuya suma sea máxima sin exceder una capacidad dada. Este problema puede verse como un caso particular del problema de la mochila 0-1 donde el peso de cada elemento es igual a su beneficio asociado[1].

Por lo tanto, para diseñar un algoritmo de aproximación que encuentre un subconjunto factible cuya suma sea mayor o igual a la suma de cualquier otro algoritmo factible se puede usar la técnica de diseño de algoritmos Greedy.

El algoritmo greedy propuesto consiste en ordenar los elementos del conjunto A de mayor a menor antes de empezar. Con el conjunto ya ordenado, se aplica la misma lógica de recorrido lineal: se evalúa elemento por elemento y, si la capacidad remanente lo permite, se lo añade al subconjunto solución; caso contrario, se lo descarta.

3.4. Garantía de calidad

Sea $A = \{a_1, a_2, \dots, a_n\}$ un conjunto de elementos enteros mayores que cero y B un número entero mayor que cero. Al ordenar el conjunto de mayor a menor, los elementos quedan dispuestos de forma tal que:

$$a_i \geq a_{i+1}, \quad \forall i \in \{1, \dots, n-1\} \quad (1)$$

Sea S el subconjunto de A creado por el algoritmo propuesto y sea T la suma de todos los elementos de S .

Si la suma de todos los elementos de A es menor o igual a B , la solución del problema es trivial: se seleccionan todos los elementos y el algoritmo devuelve una solución óptima con error nulo. Caso contrario, existirá necesariamente un elemento $a_k \in A$ que sea el primer elemento rechazado por el algoritmo debido a que su inclusión violaría la condición de factibilidad. Por lo tanto, se cumple que:

$$T + a_k > B \quad (2)$$

Dado que el conjunto se encuentra ordenado de mayor a menor según la ecuación 1, y que el elemento a_k fue rechazado, significa que todos los elementos ya presentes en S son mayores o iguales a a_k . Como el conjunto solución S no está vacío en este escenario no trivial, se deduce que:

$$T \geq a_k \quad (3)$$

Combinando las relaciones 2 y 3, se puede demostrar por sustitución que:

$$T + a_k > B \Rightarrow T + T > B \Rightarrow 2T > B \Rightarrow T > \frac{B}{2} \quad (4)$$

Dado que la solución óptima absoluta T_{opt} está acotada superiormente por B ($T_{opt} \leq B$), la relación 4 demuestra que $T > T_{opt}/2$. Con esto queda demostrada teóricamente una garantía de aproximación de 0,5 para el algoritmo propuesto.

3.5. Pseudocódigo

```
subconjunto_factible(A, B):
    si la suma de todos los elementos de A es menor o igual a B
        devolver A

    sea A' el conjunto A ordenado de mayor a menor
    sea T un numero entero inicializado en 0
    sea S un conjunto vacio

    para cada a elemento en A'
        si T + a es menor o igual a B
            guardamos a en S
            acumulamos a en T

    devolvemos S
```

3.6. Estructuras de datos utilizadas

Para la implementación de este algoritmo en Python la mejor estructura de datos a utilizar es una lista. Sólo se usa para almacenar el conjunto original y el conjunto factible encontrado, para el resto de variables se usan enteros.

3.7. Seguimiento

```
A = {3, 6, 5} ; B = 8

suma de los elementos en A: > B -> no devolvemos

A' = {6, 5, 3}
T = 0
S = {}

(i=1)
    a_i = 6  T + a_i < B -> T = 6 ; S = { 6 }
(i=2)
    a_i = 5  T + a_i > B -> T = 6 ; S = { 6 }
(i=3)
    a_i = 3  T + a_i > B -> T = 6 ; S = { 6 }

Devolver { 6 }
```

Para este ejemplo en particular la solución óptima es $\{5, 3\}$ con un $T_{max} = 8$, el algoritmo propuesto devuelve $\{6\}$ con un $T = 6$ por lo tanto $T > T_{max}/2$ y se cumple la garantía de 0,5 .

3.8. Complejidad

La complejidad de este algoritmo se puede definir según las operaciones realizadas:

- Sumar todos elementos para verificar si es un caso trivial: $O(n)$
- Ordenar la lista: $O(n \log n)$
- Recorrer la lista: $O(n)$ - Agregar los elementos a la solución: $O(1)$

$$O(n) + O(n \log n) + O(n) * O(1) \implies O(n \log n) \quad (5)$$

Con esto se concluye que la complejidad teórica del algoritmo propuesto es $O(n \log n)$.

3.9. Set de datos y tiempos de ejecución

Para los set de datos generados se tuvieron en cuenta distintos tipos de casos:

- Casos triviales: los casos donde la suma de todos los elementos dentro del conjunto A es menor o igual a B se consideran triviales porque el algoritmo simplemente realiza la suma y devuelve el mismo set recibido. Para generar estos set de datos se generó en cada uno un conjunto de elementos random y se determinó B como la suma de todos los elementos del conjunto $+ 1$.
- Casos generales: estos casos son los casos donde la suma de los elementos de A es mayor que B , en estos casos el algoritmo debe realizar un ordenamiento y recorrer la lista completa. Para generar estos casos se generó en cada uno un conjunto de números aleatorios donde cada elemento puede tomar valor en un rango de 1 a n^2 para permitir mayor variedad de elementos. El valor B de estos casos se determinó como la suma de un subconjunto elegido al azar multiplicado por otro valor aleatorio entre 0,5 y 0,8 para garantizar una solución factible pero no necesariamente exacta.
- Casos aleatorios: para sumar casos de interés se realizaron set de datos donde cada valor podía tomar valores aleatorios entre 1 y B , donde B también es elegido al azar entre 500 y 1000.

Al analizar los resultados obtenidos en los casos anteriores se observó que la diferencia entre la suma total del conjunto obtenido y el valor B era siempre muy pequeña o incluso nula. Al analizar los set de datos generados se comprobó que el problema era que a mayor tamaño del conjunto la variedad de los elementos no aumentaba, por lo tanto al ordenar los elementos podíamos obtener conjuntos donde la diferencia entre un elemento y el siguiente era muy pequeña, al contar con muchos elementos y muy poca variedad el algoritmo greedy siempre encontraba casos muy cercanos al óptimo.

Por ejemplo, un conjunto generado para el caso general es $A = \{23, 3, 38, 10, 14, 29, 39, 31, 37, 11\}$ y $B = 63$. El conjunto obtenido por el algoritmo propuesto será $\{39, 23\}$ con un valor de 62.

Sin embargo, para ver casos un poco más interesantes lo ideal sería contar con un set de datos como el siguiente: $A = \{2584, 1322, 1321, 2577, 2575, 2581, 1323, 12, 17, 1325\}$ y $B = 5040$. En este caso el conjunto obtenido por el algoritmo propuesto será $\{2584, 1325, 17, 12\}$ con un valor de 3938.

Para poder generar estos casos más interesantes se tomó en cuenta la propiedad del problema que permite reducirlo a un problema de la mochila donde los pesos de cada elemento son iguales a los beneficios que proporciona. Siendo este problema clásico uno de los más estudiados, se pueden encontrar en internet múltiples set de datos generados específicamente para cada variante del problema.

En particular, para las pruebas de este trabajo se utilizó una variante del generador de instancias compartido por Jookan[3] para agregar un caso "Hard Disperse" donde los valores estén más dispersos y sea más difícil obtener una solución óptima.

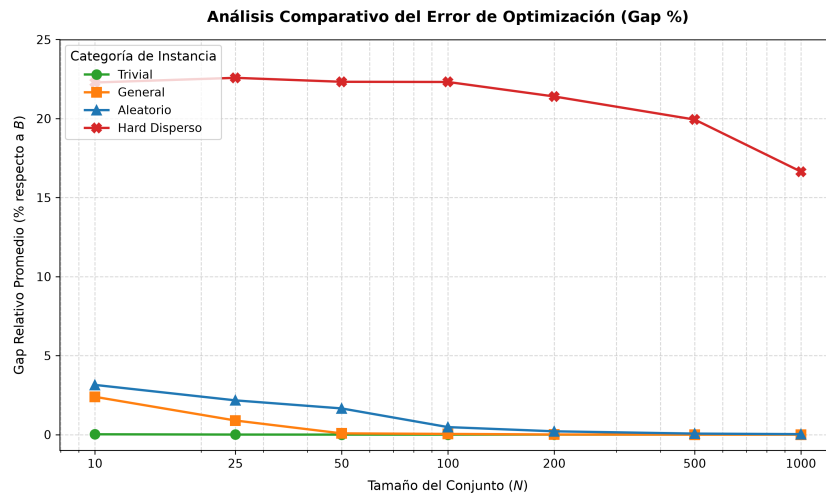
Las modificaciones realizadas consisten principalmente en las propiedades del problema a resolver, se redujo la dimensionalidad del conjunto generado dado que no es necesario tener un (peso, valor) sino simplemente el valor. Además se descartan los números mayores a B porque el algoritmo implementado no los tomará en cuenta.

En la tabla se puede apreciar lo mencionado sobre el gap promedio para las instancias creadas mediante el uso de números pseudo-aleatorios contra el obtenido para instancias creadas por el algoritmo de Jookan.

Sin embargo, para todos los casos se cumple la garantía de calidad del 50 %. Incluso vemos que ningún caso supera el 25 %, lo que nos permite concluir en que el algoritmo propuesto es una buena aproximación al óptimo considerando su simplicidad y rapidez. En el siguiente gráfico se comparan las aproximaciones del algoritmo según el tamaño del conjunto y el tipo de instancia.

Cuadro 2: Análisis comparativo de performance y calidad de optimización según el tipo de instancia ($N \in \{10, 100, 1000\}$).

Tipo de Instancia	N	Distancia promedio (%)	Tiempo (μs)
Trivial	10	0,02 %	1,05
	100	0,00 %	2,34
	1000	0,00 %	10,30
Aleatorio	10	3,16 %	2,05
	100	0,50 %	7,63
	1000	0,03 %	84,64
General	10	2,95 %	2,39
	100	0,05 %	10,68
	1000	0,00 %	92,70
Hard Disperso	10	22,29 %	2,81
	100	22,32 %	8,58
	1000	16,79 %	101,19

Figura 4: Aproximación al óptimo según tipo de instancia respecto al tamaño n del conjunto

Algo interesante a destacar es que para los casos triviales se pudo comprobar que el algoritmo siempre encontrará la solución óptima dado que devuelve el conjunto entero.

En cuanto a la complejidad temporal, teóricamente se concluyó en que el algoritmo tiene una complejidad de $O(n \log n)$ y en el siguiente gráfico se demuestra que las curvas obtenidas validan esta conclusión.

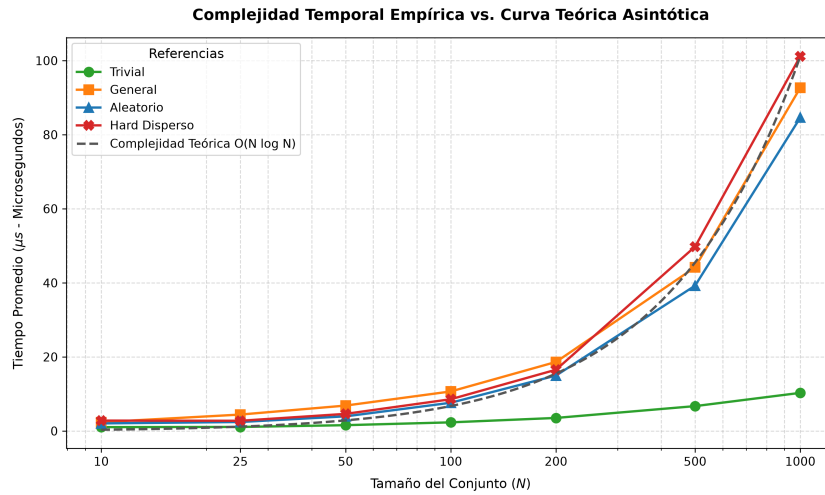


Figura 5: tiempo empírico promedio contra tiempo teórico según tipo de instancia

Nuevamente la instancia de casos triviales difiere de la complejidad temporal de las demás instancias dado que no ordena el conjunto antes de devolverlo, por lo tanto su complejidad solo consiste en la suma de todos los elementos que implica una complejidad de $O(n)$

3.10. Resultados

A partir del análisis teórico y experimental realizado para el algoritmo Greedy implementado, se puede concluir:

1. **Eficiencia y Escalabilidad Temporal:** Se comprobó empíricamente que la complejidad temporal del algoritmo está estrictamente ligada al costo de ordenamiento de los datos ($O(N \log N)$). La etapa lineal de acumulación greedy no introduce penalizaciones adicionales observables. Esto permite que el algoritmo sea sumamente eficiente y procese instancias de mil elementos en menos de $105\mu s$, de manera independiente a la dispersión de los datos.
2. **Sensibilidad a la Densidad del Set de Datos:** La precisión de la aproximación es altamente dependiente de la estructura del dataset. En entornos densos, donde existen múltiples elementos pequeños, el algoritmo muestra una tasa de acierto excelente cercana al óptimo global (0 % de gap). Por el contrario, ante conjuntos dispersos estructurados mediante clases exponenciales fraccionarias (*Hard Disperso*), el algoritmo se estanca en óptimos locales más distantes, sosteniendo errores de optimización persistentes de hasta un 22,3 %.
3. **Cumplimiento de las Garantías Teóricas:** Se validó holgadamente la cota teórica de calidad demostrada de 0,5. Incluso en las peores condiciones de estrés numérico diseñado, el gap relativo de error no superó el umbral del 23 %, ratificando que el enfoque representa una solución balanceada la velocidad de cómputo es prioritaria frente a la necesidad de optimalidad absoluta en problemas NP-completos.

4. Problema 4 – Algoritmos Aleatorios

4.1. Problema a Resolver

Dado un grafo no dirigido $G = (V, E)$, se busca un algoritmo que asigne a cada vértice uno de tres colores posibles, de manera que se maximice la cantidad de aristas "satisfechas" (aristas cuyos dos extremos tienen colores distintos). Si c^* es la cantidad máxima de aristas que puede satisfacer cualquier 3-coloreo (óptimo), se pide un algoritmo aleatorio de tiempo polinomial cuyo valor esperado de aristas satisfechas sea al menos $(2/3) \cdot c^*$.

4.2. Supuestos

- El grafo $G = (V, E)$ es simple (sin lazos ni aristas múltiples) y no dirigido.
- Los colores se representan con los valores 0, 1, 2.
- Se considera que cada vértice recibe su color de forma independiente del resto, mediante un sorteo uniforme entre los 3 colores disponibles.
- Se asume que generar un número aleatorio uniforme en 0, 1, 2 es una operación de tiempo constante $O(1)$.

4.3. Algoritmo y Estructura de Datos

La idea central es asignar a cada vértice un color elegido al azar, de forma uniforme e independiente entre los tres colores disponibles. Luego se recorren las aristas y se cuenta cuántas quedaron satisfechas.

Estructuras de datos utilizadas:

- Lista de aristas (E): se representa el grafo como una lista de pares (u, v) , suficiente porque el algoritmo solo necesita recorrer las aristas una vez.
- Arreglo `color[1..n]`: arreglo de tamaño $|V|$ donde se almacena el color asignado a cada vértice (acceso $O(1)$).
- Contador de aristas satisfechas: variable entera que acumula el resultado.

4.4. Pseudocódigo

A continuación, se presenta el pseudocódigo general del algoritmo implementado:

`coloreo_aleatorio(grafo  $G=(V,E)$ )`

```
para cada vertice v en v:
    asignar_color_aleatorio[v]

satisfechas = 0

para cada arista (u,v) en E:
    si el color[u] != color[v]
        satisfechas++

return satisfechas
```

4.5. Seguimiento de Ejemplo

Se utiliza como ejemplo el grafo completo K_4 : 4 vértices 0, 1, 2, 3 donde todos están conectados entre sí, dando 6 aristas: (0,1), (0,2), (0,3), (1,2), (1,3), (2,3). Como hay 4 nodos pero solo 3 colores disponibles, es imposible que todos los nodos tengan colores distintos: sí o sí dos nodos van a compartir color, y la arista entre ellos no quedará satisfecha. Lo mejor que se puede lograr es satisfacer 5 de las 6 aristas, por lo tanto $c^* = 5$ y $(2/3) \cdot c^* \approx 3,33$. Se muestran a continuación tres corridas del algoritmo sobre el mismo grafo para ilustrar que el resultado varía en cada ejecución:

Corrida 1

1. Soreto de Colores:

- `color[0] = 0`
- `color[1] = 1`
- `color[2] = 2`
- `color[3] = 0`

2. Recorrido de Aristas:

- (0,1): color[0]=0, color[1]=1 satisfechas = 1
- (0,2): color[0]=0, color[2]=2 satisfechas = 2
- (0,3): color[0]=0, color[3]=0 NO satisfecha. satisfechas = 2
- (1,2): color[1]=1, color[2]=2 satisfechas = 3
- (1,3): color[1]=1, color[3]=0 satisfechas = 4
- (2,3): color[2]=2, color[3]=0 satisfechas = 5

3. Resultado: satisfechas = 5 = c*

Corrida 2

1. Sorteo de Colores:

- color[0] = 0
- color[1] = 0
- color[2] = 1
- color[3] = 0

2. Recorrido de Aristas:

- (0,1): color[0]=0, color[1]=0 NO satisfecha. satisfechas = 0
- (0,2): color[0]=0, color[2]=1 satisfechas = 1
- (0,3): color[0]=0, color[3]=0 NO satisfecha. satisfechas = 1
- (1,2): color[1]=0, color[2]=1 satisfechas = 2
- (1,3): color[1]=0, color[3]=0 NO satisfecha. satisfechas = 2
- (2,3): color[2]=1, color[3]=0 satisfechas = 3

3. Resultado: satisfechas = 3.

Corrida 3

1. Sorteo de Colores:

- color[0] = 1
- color[1] = 1
- color[2] = 1
- color[3] = 2

2. Recorrido de Aristas:

- (0,1): color[0]=1, color[1]=1 NO satisfecha. satisfechas = 0
- (0,2): color[0]=1, color[2]=1 NO satisfecha. satisfechas = 0
- (0,3): color[0]=1, color[3]=2 satisfechas = 1
- (1,2): color[1]=1, color[2]=1 NO satisfecha. satisfechas = 1
- (1,3): color[1]=1, color[3]=2 satisfechas = 2
- (2,3): color[2]=1, color[3]=2 satisfechas = 3

3. Resultado: satisfechas = 3.

4.6. Demostración

Se quiere demostrar que el valor esperado de aristas satisfechas cumple:

$$E[\text{satisfechas}] \geq (2/3) \cdot c^*$$

Para cada arista $e = (u, v) \in E$, se define la variable aleatoria indicadora:

- $X_e = 1$ si $\text{color}[u] \neq \text{color}[v]$
- $X_e = 0$ si $\text{color}[u] = \text{color}[v]$

El total de aristas satisfechas es entonces:

$$\text{satisfechas} = \sum_{e \in E} X_e$$

Por construcción del algoritmo, $\text{color}[u]$ y $\text{color}[v]$ son variables independientes, cada una uniforme sobre $\{0, 1, 2\}$, con probabilidad $1/3$ para cada color. Fijado el color de u , la probabilidad de que $\text{color}[v]$ resulte distinto es $2/3$, porque hay 2 sobre 3 colores posibles para v .

$$P(X_e = 1) = P(\text{color}[u] \neq \text{color}[v]) = \frac{2}{3}$$

y por lo tanto:

$$E[X_e] = P(X_e = 1) = \frac{2}{3}, \quad \text{para toda arista } e \text{ del grafo.}$$

Como $\text{satisfechas} = \sum_{e \in E} X_e$, por la linealidad de la esperanza:

$$E[\text{satisfechas}] = E\left[\sum_{e \in E} X_e\right] = \sum_{e \in E} E[X_e] = \sum_{e \in E} \frac{2}{3} = \frac{2}{3}m$$

Por definición, c^* es la máxima cantidad de aristas que puede satisfacer cualquier coloreo posible. Ningún coloreo, ni siquiera el óptimo, puede satisfacer más aristas de las que tiene el grafo. Por lo tanto:

$$c^* \leq m$$

Entonces:

$$\frac{2}{3}c^* \leq \frac{2}{3}m$$

Por lo tanto:

$$E[\text{satisfechas}] = \frac{2}{3}m \geq \frac{2}{3}c^*$$

4.7. Análisis de Complejidad

Sea $n = |V|$ y $m = |E|$.

- Asignación de colores: se recorre una sola vez el conjunto de vértices, realizando una operación $O(1)$ (sorteo y asignación) por cada uno $\rightarrow O(n)$.
- Evaluación de aristas: se recorre una sola vez el conjunto de aristas, realizando una comparación $O(1)$ por cada una $\rightarrow O(m)$.

Complejidad: $O(n + m)$

4.8. Informe de Resultados

Se ejecutó el algoritmo aleatorio de coloreo sobre 4 sets de datos distintos, corriendo cada 50.000 veces y promediando la cantidad de aristas satisfechas. La siguiente tabla resume los resultados obtenidos:

Dataset	Vértices (n)	Aristas (m)	c^*	Cota ($\frac{2}{3}c^*$)	Promedio	Cumple
dataset_bipartito_k33.csv	6	9	9	6.000	6.011	SÍ
dataset_k4.csv	4	6	5	3.333	4.010	SÍ
dataset_k5.csv	5	10	8	5.333	6.670	SÍ
dataset_random10.csv	10	17	17	11.333	11.333	NO

Análisis por dataset

- K3,3 Bipartito: Al tratarse de un grafo bipartito, existe un 3-coloreo que satisface absolutamente todas las aristas, por lo que $c^* = m = 9$. El promedio (6.011) superó levemente la cota pedida (6.000), confirmando el resultado teórico en el caso límite donde la cota $(2/3)c^*$ coincide exactamente con $(2/3)m$.
- K4 y K5: En ambos casos, al no ser 3-coloreables de forma perfecta, c^* quedó por debajo de m (5 de 6 aristas en K4, 8 de 10 en K5). El algoritmo superó la cota en K4, el promedio (4.01) superó a $(2/3)c^*$ en más de un 20 %, y en K5 el promedio (6.67) superó la cota en más de un 25 %. Esto es consistente con lo esperado: cuanto más difícil es el grafo de colorear (mayor brecha entre c^* y m), mayor es el margen de sobrecumplimiento, ya que la garantía real del algoritmo está atada a $(2/3)m$ y no a $(2/3)c^*$.
- Random10: Al ser $c^* = m = 17$, el grafo resultó ser perfectamente 3-coloreable, y la cota pedida coincide exactamente con $(2/3)m = 11,333$. La cota y el promedio aparecen iguales, esto se debe a un redondeo previo a la muestra de los resultados, pero la comparación se realiza previo al redondeo.

¿Se cumple con el valor esperado inicialmente? Si, se cumple. Se demuestra que: $E[\text{satisfechas}] = (2/3) \cdot m \geq (2/3) \cdot c^*$ con igualdad exacta cuando $c^* = m$.

Referencias

- [1] Martello, S., & Toth, P. (1990). Knapsack Problems: Algorithms and Computer Implementations. John Wiley & Sons.
- [2] Cornell University. (2010). The Edmonds-Karp max-flow algorithm. CS 4820: Introduction to Algorithms. Recuperado de <https://www.cs.cornell.edu/courses/cs4820/2011sp/handouts/edmondskarp.pdf>
- [3] Jookan, J. (2020). Knapsack Problem Instances Generator. GitHub Repository. Recuperado de <https://github.com/JorikJookan/knapsackProblemInstances>